

1. Node.js Overview

A JavaScript runtime on **Chrome V8**, for running JS on the server and building modern apps.

2. Benefits of Node.js

- ✓ Full-stack Javascript (Frontend & Backend)
- ✓ Blazing fast execution (V8)
- ✓ Efficient non-blocking I/O
- ✓ Lightweight & Event-driven
- ✓ Massive package ecosystem (NPM)
- ✓ Scalable for APIs & real-time applications

3. The V8 Engine

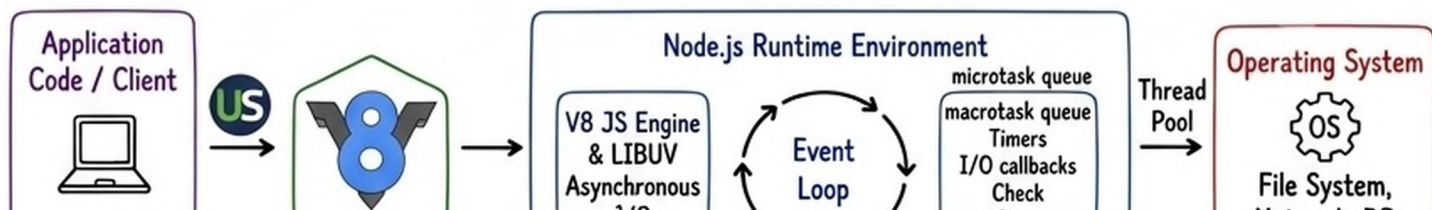
Google's open-source powerhouse. Directly translates JS into native machine code, providing exceptional speed.



4. Comparing Node.js & Express.js

Runtime Environment	Backend Framework
Core execution environment	Structure and tools for APIs
Provides fundamental APIs	Adds a layer over Node
Built-in node features	Advanced routing and middleware
Example: A basic server	Example: Express-based API

10. Node.js Architecture (Conceptual)



5. Key Architectural Principles

- Event-driven & non-blocking I/O model
- Single main thread with Event Loop
- Asynchronous operations
- Excellent concurrent connection handling
- Native scalability

6. Non-Blocking for Performance

Utilizes a **single thread** to delegate I/O tasks (file, network) to system-level threads via **'libuv'**, continuing main loop operations without delay.

7. Primary Use Cases

- 🌐 Traditional Web Servers
- 💬 Real-time Chats & Gaming (Socket.io)
- 📱 Building RESTful APIs
- 📺 Streaming Platforms (Video/Audio)
- ☁ Serverless & Lambda Functions
- 🔗 Microservices Architecture

8. Pros & Advantages

- ✓ High performance for I/O
- ✓ Excellent scalability
- ✓ Thriving NPM ecosystem
- ✓ Unified full-stack language
- ✓ Ideal for modern agile development

9. Cons & Challenges

- ✗ CPU-intensive tasks (due to single thread)
- ✗ Risk of callback hell (historical, mitigate with async/await)
- ✗ The single-thread performance bottleneck
- ✗ Occasional api instability with version updates

Node.js Event Loop

Deep Dive & Mastery

☆ Mastering Node.js ☆
Concurrency!

Event Loop Diagram

1. Call Stack

- It is a stack data structure.
- It stores the current execution context.
- JS is single threaded, so only one operation goes into call stack at a time.

2. Callback Queue

- Stores callback functions ready to be executed.
- Also called Task Queue.

3. Priority Processing Queue

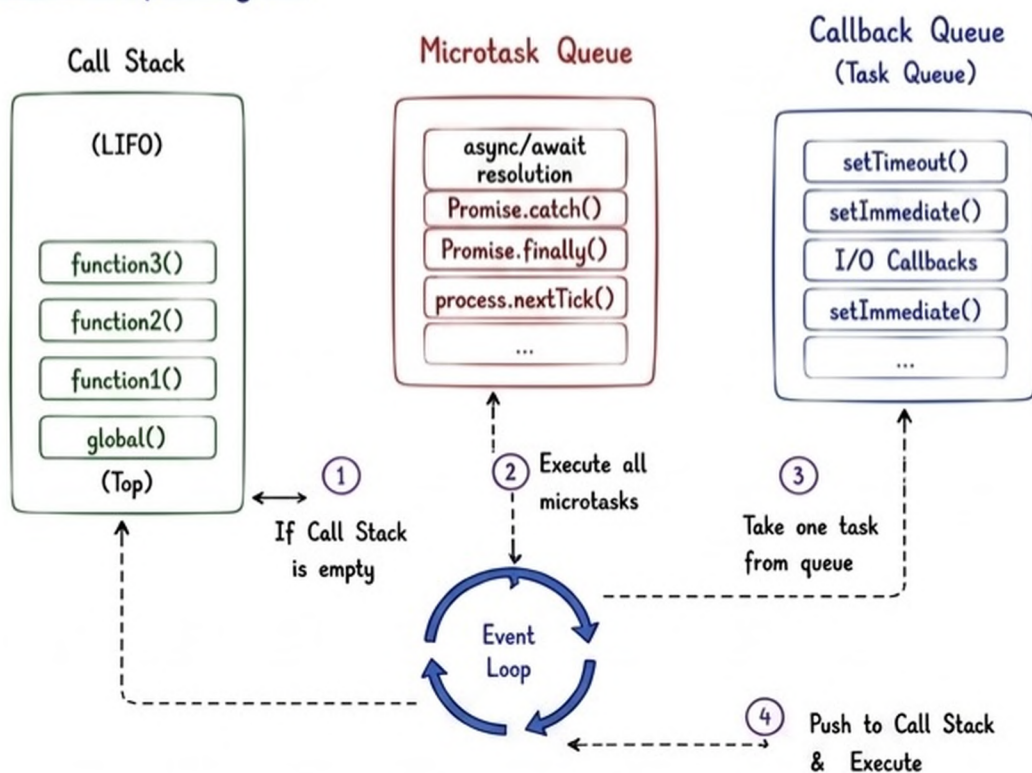
- Stores microtasks like Promises, `process.nextTick()`.
- Has higher priority than Callback Queue.

4. Event Loop

- Continuously checks call stack.
- If call stack is empty:
 - Executes all microtasks.
 - Then takes one task from callback queue and pushes it to call stack.

5. Promises

- Handled in Microtask Queue.
- `.then()`, `.catch()`, `.finally()` are executed as microtasks.



Timers Comparison

Phase of Execution	Synon Phase	Callback Phase	NextTick Queue
<code>setTimeout</code>	Executes minimum - highest priority.	Tasks immediately after microtasks.	Executes Priority Processing Queue.
<code>setInterval()</code>	Makes current task is only.	Goes to Callback Queue.	NextTick.
I/O Callbacks	Goes to <code>async/await</code> resolution.	Added to Callback priority.	Run a Callback queue.
<code>setImmediate()</code>	Runs before the event priority.	Use with highest priority.	Not a pause & execute.

Output Order

- | | | |
|-----------------|---|--------------------------------|
| 1. Begin | } | Synchronous (Call Stack) |
| 2. End | | |
| 3. nextTick | → | (Microtask - highest priority) |
| 4. Promise | → | (Microtask) |
| 5. setImmediate | → | (Callback Queue) |
| 6. Timeout | → | (Callback Queue - after delay) |

Example Flow

```
console.log("Begin");
setImmediate(() => console.log("I/O Check", 0));
process.nextTick(() => console.log("Tick A"));
Promise.resolve().then(() => console.log("Promise A"));
setTimeout(() => console.log("Timer A", 1), 1);
console.log("End");
```

Node.js Core Modules

@codeera.tech

★ The Essential Built-in Modules in Node.js ★



Interview Tip

Core modules are highly optimized and written in C/C++, designed for peak performance.

1. fs (File System)

- Direct control over the file system.
- Create, write, update, read, or delete files.

```
const fs = require('fs');
// Read file
const data = fs.readFileSync(
  'readme.txt', 'utf8');
console.log(data);
// Synchronous read
```

2. path

- Utilities for working with file and directory paths.

```
const path = require('path');
console.log(path.join(__dirname,
  'folder', 'file.txt'));
console.log(path.basename(__filename));
console.log(path.extname(__filename));
```

3. http

- Create HTTP server & make HTTP requests.

```
const http = require('http');
const server = http.createServer(
  (req, res) => {
    res.writeHead(200, {'Content-Type':
      'text/plain'});
    res.end('Hello Nod.js');
  });
server.listen(3000);
```

4. https

- Same as http but with SSL/TLS (secure).

```
const https = require('https');
https.get('https://example.com',
  (res) => {
    console.log(res.statusCode);
  });
```

5. os

- OS Platform & Architecture info
- System uptime & memory statistics

```
const os = require('os');
console.log(os.platform());
console.log(os.spoo().length);
console.log(os.totalmem());
console.log(os.freemem());
```

6. crypto

- Provides cryptographic functionality like hashing, encryption, etc.

```
const crypto = require('crypto');
const secret = 'codeera';
const hash = crypto.createHmac('sha256',
  .update('hello')
  .digest('hex'));
console.log(hash);
```

7. stream

- Handle streaming data efficiently.
- Useful for large files.

```
const fs = require('fs');
const readStream = fs.createReadStream(
  'source.zip');
const writeStream = fs.createWriteStream(
  'dest.zip');
readStream.on('data', chunk =>
  writeStream.write(chunk));
// Manual chunking
```

8. buffer

- Handle binary data directly.
- Useful for working with raw data.

```
const buf = Buffer.from('Node.js');
console.log(buf); // <Buffer ...>
console.log(buf.toString()); // Node.js
console.log(buf.length); // length
```

9. events (EventEmitter)

- Create, fire and listen for events.
- Useful for event driven programming.

```
const EventEmitter = require('events');
const emitter = new EventEmitter();
emitter.on('greet', (name) => {
  console.log('Hello ' + name);
});
emitter.emit('greet', 'Abhi');
```

10. process

- Provides information about & control over the Node.js process.

```
console.log(process.pid); // Process ID
console.log(process.platform); // Platform
console.log(process.env); // Environment variables
process.exit(0); // Exit process
```

Additional Important Modules

url
querystring
util
events
zlib
timers
child_process

Parse and format URL.
Parse URL query string.
Utility functions like format, inspect, etc.
(Already covered) Event handling.
Compression / decompression.
setTimeout, setInterval (global but part of timers).
Spawn new processes.

Express.js Complete Guide

★ The Most Popular Node.js Framework ★

@codeera.tech



Interview Tip

Express.js is a minimal and flexible framework for building web applications and APIs

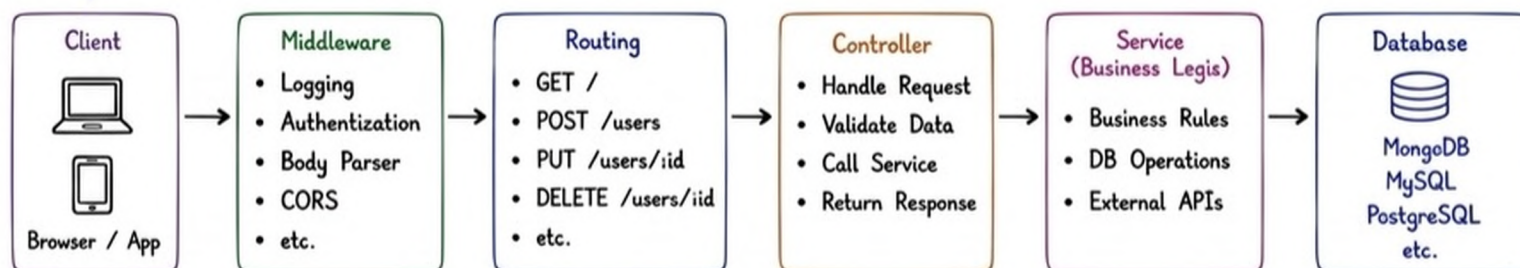
1. What is Express.js?

Express.js is a fast, unopinionated, minimalist web framework for Node.js. It helps us build web apps and APIs easily.

2. Why Express.js?

- ✓ Minimal and flexible
- ✓ Easy to learn and integrate
- ✓ Powerful routing
- ✓ Middleware support
- ✓ Huge ecosystem (NPM)
- ✓ Perfect for REST APIs

3. Express Architecture



4. Routing

```
const express = require('express');
const app = express();

app.get('/', (req, res) => res.send('Home'));
app.post('/users', (req, res) => res.send('User Created'));
app.put('/users/:id', (req, res) => res.send('User Updated'));
app.delete('/users/:id', (req, res) => res.send('User Deleted'));
```

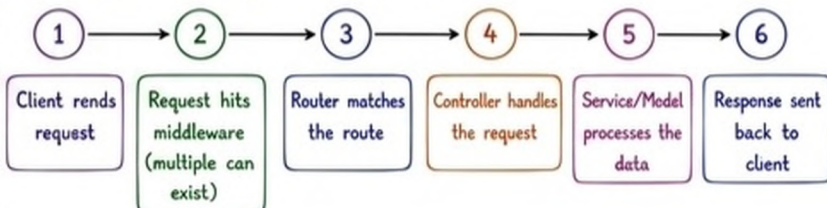
5. Middleware

- Functions that have access to req, res, and next() in the request-response cycle.
- Can execute code, modify request/response, and cycle or call next().

```
// Built-in Middleware
app.use(express.json()); // Parse JSON body
app.use(express.urlencoded({ extended: true }));
app.use(express.static('public')); // Serve static files

// Custom Middleware
const logger = (req, res, next) => {
  console.log(`${req.method} ${req.url}`);
  next();
};
app.use(logger);
```

6. Request Lifecycle in Express



7. Error Handling

Use error handling middleware with 4 parameters. (err, req, res, next) at the end.

```
app.use((err, req, res, next) => {
  console.error(err.stack);
  res.status(500).json({
    success: false,
    message: 'Something went wrong!'
  });
});
```

8. Folder Structure (Best Practice)

```
my-app/
├── src/
│   ├── config/      # DB, env, config
│   ├── controllers/ # Request handlers
│   ├── routes/      # Route definitions
│   ├── services/    # Business logics
│   ├── models/      # DB models
│   ├── middlewares/ # Custom middlewares
│   ├── utils/       # Helper functions
│   └── app.js       # Express app setup
└── .env
```

9. REST API Design

Method	Endpoint	Description	Example
GET	/api/users	Retrieve all user accounts	Retrieve ID 10
GET	/api/users	Get user by id	GET /users/1
POST	/api/users	Create new user	POST /users
PUT	/api/users	Update user	PUT /users/1
PATCH	/api/users	Partial update	PATCH /users/1

Best Practices

- ✓ Routes focused on one function
- ✓ Clear meaningful route names
- ✓ Validate input data
- ✓ Robust data validation
- ✓ Proper HTTP gracefully
- ✓ Keep business logic in services
- ✓ Don't repeat yourself (DRY)
- ✓ Environment based configs
- ✓ Enable CORS securely
- ✓ Use 'helmet' for secure headers

★ Defensive Measures to Shield Your APIs ★



Interview
Always add
principle of
trust.

1. Understanding JWT (JSON Web Tokens)

- Stateless token issuance.
- Tokens are digitally signed using a key or secret.
- Client includes token in the Authorization header.



```

// Generate Token
const token = jwt.sign({ id: user.id },
  process.env.JWT_SECRET, { expiresIn: '7d' });
// Verify Token
const decoded = jwt.verify(token,
  process.env.JWT_SECRET);
  
```

2. Cookies

- Small data stored in browser.
- Sent automatically with every request.
- Can be used to store session or JWT.
- Use `HttpOnly` and `secure` flag.

```

res.cookie('token', token, {
  httpOnly: true,
  secure: true,
  sameSite: 'strict',
  maxAge: 7 * 24 * 60 * 60 * 1000
});
  
```

3. Sessions

- Stores session data on server.
- Session ID stored in cookie.
- More secure but stateful.
- Requires store like Redis / DB.

```

req.session.userId = user.id;
req.session.isLoggedIn = true;

// Access
req.session.userId;
  
```

4. OAuth 2.0

- Authorization framework.
- Allows users to login using Google, GitHub, Facebook, etc.
- Uses access token.

```

passport.use(new GoogleStrategy({
  clientID: GOOGLE_ID,
  clientSecret: GOOGLE_SECRET,
  callbackURL: '/auth/google/callback',
  verifyCallback()
}));
  
```

5. bcrypt (Password Hashing)

- Never store passwords in plain text.
- Use bcrypt to hash passwords.
- Strong and slow hashing algorithm.

```

const bcrypt = require('bcrypt');
// Hash Password
const hash = await bcrypt.hash(password, 10);
// Compare Password
const isMatch = await bcrypt.compare(
  password, hash);
  
```

6. Helmet (Secure Headers)

- Helps set various HTTP headers to secure app.
- Protects against common attacks.

```

const helmet = require('helmet');
app.use(helmet());

// It sets headers like:
// X-Content-Type-Options, X-Frame-Options,
// X-XSS-Protection, Strict-Transport-Security
  
```

7. Preventing CORS & CSRF

- Allows or restricts resources on different origin.
- Important for APIs.

```

const cors = require('cors');
app.use(cors({
  origin: 'https://your-frontend.com',
  methods: ('GET', 'POST', 'PUT', 'DELETE')
}));
  
```

8. CSRF (Cross Site Request Forgery)

- Attack that tricks user to perform unwanted actions.
- Mitigation Use CSRF tokens.

```

const csrf = require('csurf');
app.use(csrf({ cookie: true }));
// Send token to client
res.render('form', { token: req.csrfToken() });
// Validate token in requests
  
```

9. Guarding against XSS

- Strict input sanitization.
- Limit template access.

```

// Use libraries like 'xss-sanitizer'
const xss = require('xss-sanitizer');
app.use(xss());
// Or escape output using template engines like EJS, Pug
  
```

10. Rate Limiting Your Tra

- Prevents abuse by limiting number of requests.
- Protects from brute force & DDoS.

```

const rateLimit = require('express-rate-limit');
const limiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 minutes
  max: 100 // max 100 requests per windowMs
});
app.use('API/', limiter);
  
```

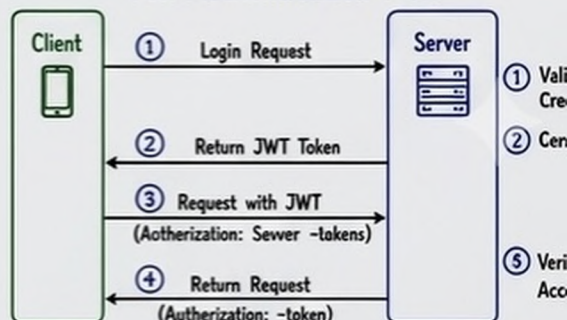
JWT vs. Session (Storage & Flow)

Feature	JWT	Session
Storage	Client (Browser)	Server (DB/Redis)
State	Stateless	Stateful
Scalability	Highly Scalable	Less Scalable
Revocation	Difficult	Easy (Destroy session)
Use Case	APIs, Microservices	Web Applications
Uses	Image	App! Server Requirement

Comprehensive Security Checklist

- ✓ Use HTTPS everywhere
- ✓ Validate and sanitize all inputs
- ✓ Hash passwords using bcrypt
- ✓ Use JWT with short expiration
- ✓ Store secrets to environment variables
- ✓ Use Helmet for secure headers
- ✓ Enable CORS with specific origins
- ✓ Add Rate Limiting
- ✓ Protect against CSRF and XSS
- ✓ Keep dependencies updated
- ✓ Keep dependencies access.

Full Authentication Flow



★ Store, manage and interact with data efficiently ★



Interview Tip

Know the database, ORM, connections, transactions and performance concepts.

1. Popular Databases



Common Database choices for Node.js



MongoDB - document-based (NoSQL)



PostgreSQL - relational (SQL)



MySQL - relational (SQL)



Others - SQLite, Oracle, etc.

2. ORM / ODM in Node.js

Tools for Data Interaction

- Object Relational Mappers (ORMs) for SQL (Sequelize, Prisma)
- Object Data Mappers (ODMs) for NoSQL (Mongoose)

3. Database Connection

```
// Establish a Mongoose database connection
const mongoose = require('mongoose');
mongoose.connect('mongodb://127.0.0.1:27017/myapp', {
  useNewUrlParser: true,
  useUnifiedTopology: true
});
// Connection error handler
.then(() => console.log('DB Connected'))
.catch(err => console.log('DB Error:', err));
```

4. Mongoose (MongoDB ODM)

```
// Configure Schema and Model
const mongoose = new mongoose.Schema({
  name: String,
  email: String,
  age: Number
});
// Create Model
const User = mongoose.model('User', userSchema);
// Insert Document
const user = new User({ name: 'Abhi', email: 'abhi@mail.com',
  age: 22 });
await user.save();
// Find Documents
const users = await User.find({ age: { $gt: 18 } });
// Update Document
const User.updateOne({ email: 'abhi@mail.com' }, { age: 23 });
// Delete Document
const User.deleteOne({ email: 'abhi@mail.com' });
```

5. Prisma (SQL ORM)

```
// Initialize Prisma Client
npx prisma init
// Example: Find Users
const { PrismaClient } = require('@prisma/client');
const prisma = new PrismaClient();
const users = await prisma.user.findMany({
  where: { age: { gt: 18 } }
});
// Create User
const newUser = await prisma.user.create({
  data: {
    name: 'Abhi',
    email: 'abhi@mail.com',
    age: 22
  }
});
```

6. Sequelize (SQL ORM)

```
const { Sequelize, DataTypes } = require('sequelize');
const sequelize = new Sequelize('mydb', 'root', 'password', {
  host: 'localhost',
  dialect: 'mysql'
});
const User = sequelize.define('User', {
  name: DataTypes.STRING,
  email: DataTypes.STRING,
  age: DataTypes.INTEGER
});
// Sync Model
await sequelize.sync();
// Create
await User.create({ name: 'Abhi', email: 'abhi@mail.com', age: 22 });
// Find
const users = await User.findAll();
```

7. Connection Pooling

- Efficiently reuse open database connections.
- Reduces latency by avoiding connection creation overhead.
- Controlled with pool size settings.



```
// In Mongoose (poolSize)
mongoose.connect(..., { poolSize: 20 });
```

8. Transactions (SQL)

- Bundle multiple operations as one atomic unit.
- Ensure data consistency.

```
// Resume Example
await prisma.transaction({
  prisma.user.create({ data: { name: 'A' } }),
  prisma.user.create({ data: { title: 'Hello',
    userId: 1 } })
});
// Sequelize Example
const t = await sequelize.transaction();
try {
  await Post.create({ name: 'A' }, { transaction: t });
  await Post.create({ title: 'Hello' }, { transaction: t });
} catch (err) {
  await t.rollback();
}
```

9. Indexing

- Significantly improve query response times.
- Reduces search time.
- Create indices on high-query fields.

```
// MongoDB
db.users.createIndex({ email: 1 });
// SQL (PostgreSQL / MySQL)
CREATE INDEX idx_email
ON users (email);
```



10. Pagination

- Implement page-based results for large datasets.
- Better performance and user experience.

```
// MongoDB
const page = 2, limit = 30;
const data = await User.find().skip((page-1)*limit)
  .limit(limit);
```

Typical Architecture



Node.js Scaling & Deployment

- Your Guide to growth

★ Techniques to build robust, distributed, and high-performance applications ★

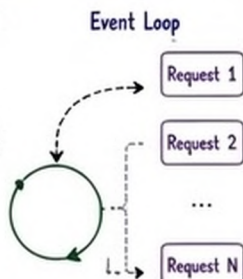


Interview Tip

Pro Tip: Leverage both horizontal and vertical scaling strategies to efficiently manage traffic and improve fault tolerance.

1. How Node.js Handles requests

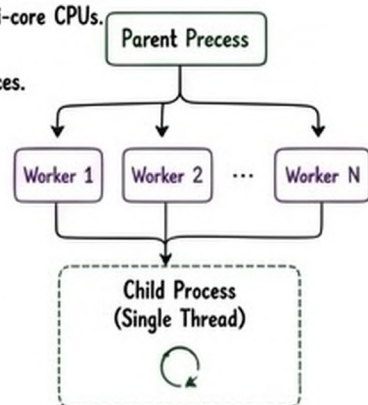
- Delegates operations to an internal system for concurrency.
- Utilizes libuv for non-blocking I/O.
- Avoid long-running tasks to prevent Event Loop blockage.



2. Using the built-in Cluster Module

- Spawns multiple Node.js processes for multi-core CPUs.
- Each process has isolated memory.
- Ideal for utilizing multi-core server resources.

```
const cluster = require('cluster');
const os = require('os');
if (cluster.isMaster) {
  const numCPUs = os.cpus().length;
  for (let i = 0; i < numCPUs; i++) {
    cluster.fork();
  }
} else {
  // Your app code
}
```



3. Process management with PM2

- Production-ready process manager.
- Automates restart on crash, log rotation.
- Provides monitoring and load balancing.

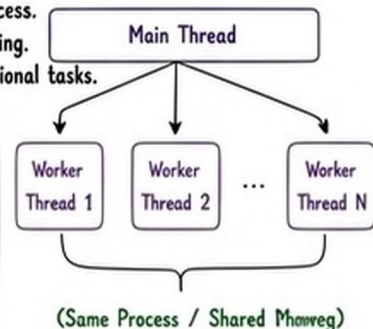
PM2

```
# Install
npm install -g pm2
# Start App
pm2 start app.js -i max      # cluster mode
pm2 list                     # list all apps
pm2 monitor                  # monitoring
pm2 restart all               # restart
pm2 logs                      # logs
```

4. Worker Threads for parallel computing

- Enables multiple threads within a single process.
- Share data via message passing, avoids locking.
- Great for offloading CPU-intensive computational tasks.

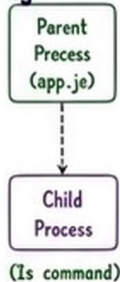
```
const { Worker } = require('worker_threads');
const worker = new Worker('./worker.js');
worker.on('message', (data) => {
  console.log(data);
});
worker.postMessage({ task: 'start' });
```



5. Child Process with detailed rephrasing

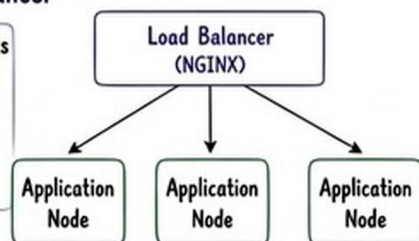
- Creates a distinct, separate system process.
- No shared memory; independent execution.
- Use for executing system commands or isolated

```
const { spawn } = require('child_process');
const process = spawn('ls', ['-la']);
process.stdout.on('data', (data) => {
  console.log(`output: ${data}`);
});
```



6. Introducing a Load Balancer

- Efficiently spreads incoming requests across multiple app instances.
- Increases system availability and throughput.
- Increases system availability and
- Examples: NGINX, HAProxy, F5.



7. Redis as Cache & Session Store

- In-memory, high-speed data structure store.
- Shared across application nodes for data persistence.
- Useful for distributed caching, session data, and leaderboards.



```
const redis = require('redis');
const client = redis.createClient();
client.set('key', 'value');
client.get('key', (err, data) => {
  console.log(data);
});
```

Use Cache:

- ✓ Caching data
- ✓ Shared Session Storage
- ✓ Rate Limiting control
- ✓ Real-time Leaderboards
- ✓ Pub/Sub messaging

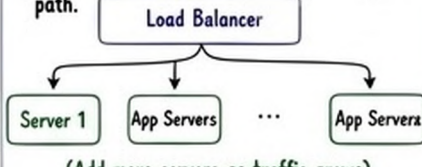
8. Decoupling with a Message Queue

- Provides asynchronous message processing for reliable delivery.
- Decouples service components.
- Handles traffic bursts and background jobs.
- Examples: RabbitMQ, Apache Kafka, BullMQ.



9. Implementing Horizontal Scaling

- Scaling out: Adding more machine nodes or virtual instances.
- Uses a Load Balancer for even distribution.
- Provides linear scalability and easier upgrade path.



★ Guide for high-performance Node.js systems ★



Interview Tip

Utilize Streams, Caching, and best practices to create robust and swift apps.

1. Streams

- Streams handle sequential data flow.
- Excellent for processing large data sets without high memory load.
- Available stream classes include Readable, Writable, and combined Duplex/Transform.



```
const fs = require('fs');
const readStream = fs.createReadStream('input.txt');
const writeStream = fs.createWriteStream('output.txt');
readStream.pipe(writeStream); // Pipe data flow
```

2. Buffers

- Buffers provide a lower-level memory region for binary data.
- Memory is assigned once and can't be resized.
- Can be manipulated like byte arrays.

Buffer (5 bytes)

72	101	108	108	111
H	e	l	l	o

```
const buf = Buffer.from('Hello');
console.log(buf); // <Buffer 48 65 6c 6c 6f>
console.log(buf.toString()); // Hello
```

3. Piping

- Piping seamlessly channels output from one stream to input of another.
- Built-in error handling and flow control.



```
fs.createReadStream('bigfile.zip')
  .pipe(clib.createGzip())
  .pipe(fs.createWriteStream('bigfile.zip.gz'));
// full flow: read, zip, write
```

4. File Upload (Using Streams)

- Store incoming file directly to disk without using storage directly to disk

```
const http = require('http');
const fs = require('fs');
const server = http.createServer((req, res) => {
  if (req.method === 'POST') {
    const fileStream = fs.createWriteStream('upload.did');
    req.pipe(fileStream);
    fileStream.on('finish', () => res.end('Upload Done'));
  }
});
server.listen(5000);
```

5. Compression

- Minimize response size.
- Reduces total request time and bandwidth.

```
const express = require('express');
const compression = require('compression');
const app = express();

app.use(compression());

app.get('/', (res, res) => {
  res.send('Welcome, Newbie!');
});

app.listen(3000);
```

6. Caching

- Retain frequently accessed data. ✓ In-Memory Cache (node-cache)
- Decreases application latency and database overhead.
- Enhances overall response times. ✓ Redis ✓ Memcached

Popular Caching Options

```
const NodeCache = require('node-cache');
const cache = new NodeCache({ stdTTL: 60 }); // 60 sec
cache.set('user:1', { id: 1, name: 'Abhi' });
const user = cache.get('user:1');
console.log(user);
```

7. Memory Leaks

- Leaks cause applications to slow and eventually fail.
- Typical culprits:
 - ✓ Global namespace pollution
 - ✓ Persistent event handlers
 - ✓ Growing object collections
 - ✓ Closures holding references

Tips

- Use leaks (heapdump, clinic.js)
- Close connections
- // Performance tools
- Unbind events

8. Garbage Collection (GC)

- Automatically reclaims memory to prevent leaks.
- VB's sophisticated GC processes (Scavenge, Mark-Sweep-Compact).
- Clean coding is essential for optimal GC performance.
- Write clean code to help GC.

Best Practices for GC

- ✓ Global global variables
- ✓ Release references
- ✓ Use streams for large data
- ✓ Avoid memory leaks

9. EventEmitter

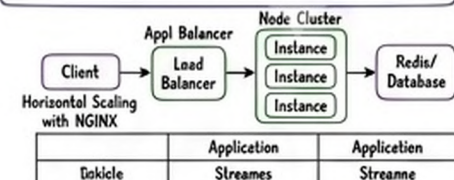
- Core module to handle events.
- Used in many Node.js modules.

Event
Emitter
(Emurce)

```
const EventEmitter = require('events');
```

10. Performance Optimization Tips

- ✓ Use Streams for large data
- ✓ Enable Compression
- ✓ Cache frequent data
- ✓ Use Connection Pooling



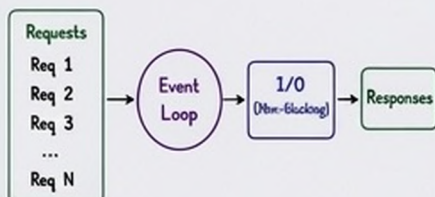


Tip for Interviews:

Prioritize concise, structured answers. Share practical scenarios and highlight performance considerations.

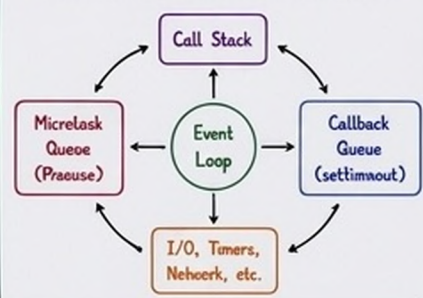
1. Why is Node.js Single-threaded?

- Node.js uses single thread with Event Loop.
- It is non-blocking I/O, so it can handle thousands of concurrent requests.
- CPU-intensive tasks can block the event loop, so use Worker Threads/Cluster.



2. Explain the Event Loop in Node.js.

- Event Loop is the heart of Node.js.
- It allows non-blocking I/O operations.
- It continuously checks Call Stack, Microtask Queue and Callback Queue.



3. process.nextTick() vs setImmediate()

process.nextTick()	setImmediate()
1. Goes to Microtask Queue	1. Goes to Check Phase (Callback Queue)
2. Executes before setImmediate()	2. Executes after I/O events
3. Can starve the Event Loop if used recursively	3. Depends on Event Loop phases
<pre>process.nextTick(() => { console.log("nextTick"); });</pre>	<pre>setImmediate(() => { console.log("immediate"); });</pre>

4. Streams vs Buffers

Streams	Buffers
- Used to read/write large data piece by piece. - Memory efficient. - Example: <code>fs.createReadStream</code>, <code>fs.createWriteStream</code>	- Used to handle small binary data. - Stores data in fixed size chunks. - Example: <code>Buffer.from()</code>
Use Streams for large files, Buffers for small data.	

5. CommonJS vs ES Modules

	CommonJS (CJS)	ES Modules (EJS)
Syntax	<code>require()</code> , <code>module.exports</code>	<code>import</code> , <code>export</code>
Loading	Synchronous	Asynchronous
Default Support	Yes	Yes
Use	Older Node.js applications	Modern Node.js applications
File Ext.	.js	.mjs or "type": "module" in package.json

6. JWT vs Session

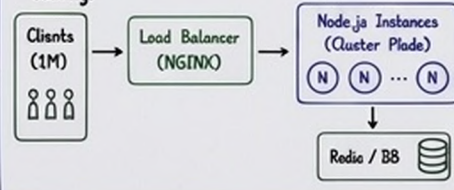
JWT	Session
- Stateless - Token stored in client (<code>localStorage</code>, cookie) - Scalable - Good for APIs & microservices	- Stateful - Data stored on server (usually in Redis/DB) - Less scalable - Good for traditional web apps
JWT = Scalability Session = Security (easy to revoke)	

7. Child Process vs Worker Threads

Child Process	Worker Threads
- Creates separate process. - Has its own memory. - Good for isolation. - More overhead.	- Runs in same process. - Shares memory (with limitations). - Good for CPU-intensive tasks. - Less overhead.
Use Child Process for isolation, Worker Threads for performance.	

8. How does Node.js handle 1 Million Concurrent Connections?

- Node.js uses non-blocking I/O and Event Loop.
- Uses OS features like `epoll` (Linux) or `kqueue` (Mac) for handling connections.
- Minimal memory per connection.
- Use Cluster / Load Balancer for horizontal scaling.



9. Why use Express.js?

- Minimal and flexible framework.
- Huge ecosystem (middlewares, plugins).
- Easy routing and middleware support.
- Fast development.
- Production ready.

```
const express = require('express');
const app = express();
app.get('/', (req, res) => res.send('Hello'));
app.listen(3000);
```

10. Middleware in Node.js / Express?

- Functions that have access to `req`, `res` and `next()`.

11. Error Handling in Express?

- Use `try/catch` for sync code.
- Use `next(err)` for async errors.
- Centralized error handling middlewares.

12. Environment Variables in Node.js?

- Keep sensitive data outside code.
- Use `.env` file and `dotenv` package.
- Set via `process.env.VARIABLE_NAME`

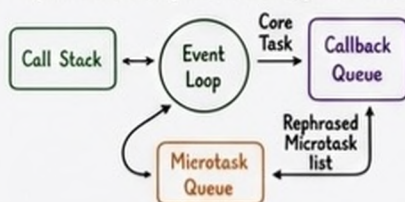


Interview Insight:

Focus on understanding architectural patterns and real-world optimizations. Prioritize robust design and quality.

1. Event-Driven Nature

- The non-blocking I/O and event loop are fundamental.
- Manages the Call Stack, Event Loop, both queues (Callback & Microtask).
- Optimizes for single-threaded high concurrency.



2. Error Handling Patterns

- `process.on('uncaughtException', (err) => {...})`
- Use try-catch and `next(err)` in Express.
- Centralized error handling middleware.

```
app.use((req, res, next) => {
  console.error(err.stack);
  res.status(err.status || 500).json({
    message: err.message || 'Something went wrong'
  });
});
```

3. Middleware Execution

- Functions that have access to `req`, `res`, `next()`.
- Executed in the order they are defined.
- Types: Built-in, Third-party, Custom.

```
const logger = (req, res, next) => {
  console.log(`${req.method} ${req.uri}`);
  next();
};
app.use(logger);
```

4. Environment Variables

- Store sensitive data in `.env` file.
- Access using `process.env`.
- Use `dotenv` package.

```
require('dotenv').config();
const PORT = process.env.PORT;
const DB_USER = process.env.DB_USER;
console.log(PORT);
```

Best Practices

- ✓ Never commit `env` file
- ✓ Use different `env` for dev, staging, prod

5. Node.js Modules

- Node.js uses CommonJS (`require`, `module exports`) by default.
- ES Modules (`export/export`) are also supported.

CommonJS	ES Modules
<pre>// math.js module.exports = { add: (a, b) => a + b };</pre>	<pre>// math.js export const add = (a, b) => a + b;</pre>
<pre>// math.js const mods = require('./math'); console.log(mods.add(2, 3));</pre>	<pre>// index.js import { add } from './math.js'; console.log(add(2, 3));</pre>

6. File System API

- Built-in module to work with files.
- Supports `sync` (non-blocking) operations.

```
const fs = require('fs');
// Read File
fs.readFile('file.txt', 'utf8', (err, data) => {
  if (err) throw err;
  console.log(data);
});
// Write File
fs.writeFile('file.txt', 'Hello Node.js', err => {
  if (err) throw err;
  console.log('File written');
});
```

7. Security Measures

- ✓ Validate and sanitize all inputs
- ✓ Use Helmet for secure headers
- ✓ Prevent SQL Injection & XSS
- ✓ Use bcrypt for password hashing
- ✓ Limit rate with `express-rate-limit`
- ✓ Keep dependencies updated



8. Database Integration

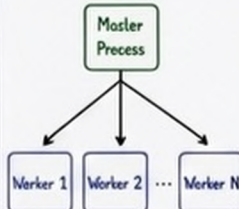
- Regular: MongoDB (NaSQL), PostgreSQL, MySQL.
- Use ODM/ORM for better abstraction.
- Example with Mongoose (MongoDB):

```
const mongoose = require('mongoose');
mongoose.transaction((err) => mongoose) => {
  // on transaction *, logic {
  console.log(err, err, error, log);
  });
};
const User = mongoose.model('User', userSchema);
```

9. Clustering & Scaling

- Utilize multi-core CPU.
- Improves performance and reliability.

```
const cluster = require('cluster');
const os = require('os');
if (cluster.isMaster) {
  const numCPUs = os.cpus().length;
  for (let i = 0; i < numCPUs; i++) {
    cluster.fork();
  }
} else {
  require('./app');
}
```



10. Caching Strategies

- Reduces DB calls and response time.
- Use: Redis, in-memory cache.

```
const redis = require('redis');
const client = redis.createClient();
const client.connect();
const client.set('key', 'value', { EX: 60 });
```

11. Logging & Observability

- Use logging libraries for better logs.
- Popular: Winston, Pino, Morgan.
- Log levels: info, warn, error, debug.

```
const winston = require('winston');
const logger = winston.createLogger({
  level: 'info',
  transports: [new winston.transports.Console()]
});
```

12. Production Process with PM2

- Keeps app alive, auto restart, load balancing.
- Monitor CPU, memory, logs.
- Commands:

```
pm2 start app.js
pm2 list
pm2 restart all
```

